

Lab 7 — Multiplayer Tic-Tac-Toe

OCSF & EventBus — Spring 2025–2026

Gal Bareket 326144300

Itai Aviad 214891665

Repository:

<https://github.com/SE-Lab-Itai-Gal/labs> (branch lab7, folder lab7/tictactoe)

1. What we built

A two-player networked **Tic-Tac-Toe** game built on the **OCSF** (Object Client–Server Framework) architecture, with a **JavaFX** graphical client and the **EventBus** (GreenRobot) Publish/Subscribe pattern.

Two players connect to the server. When the second player arrives, the server **randomly** assigns the symbols X/O and randomly decides who plays first, then the two players take turns. Every move is validated and broadcast by the server, so the two boards always stay perfectly in sync.

The project is a Maven multi-module project (mirroring the OCSF mediator example):

- **entities** — the shared, serializable protocol: `GameMessage` + `MessageType`.
- **server** — the OCSF server (`ocsf/*`), `GameServer` (game logic) and `App` (entry point).
- **client** — the JavaFX app (`App`, `Main`, `board.fxml`), the OCSF client (`GameClient`), the `BoardController` and the `GameEvent`.

2. How it works

The server is the single source of truth. A client never draws a move on its own — it asks the server, the server checks that it is that player’s turn and the cell is free, applies the move, and *echoes* the validated move back to *both* players. This is what guarantees the two GUIs cannot diverge.

Networking and UI are decoupled with EventBus. The OCSF client (`GameClient`) receives objects from the server and re-publishes them on the `EventBus`. The `BoardController` is a subscriber (`@Subscribe`) and updates the JavaFX scene on the FX thread (`Platform.runLater`). The controller therefore never touches the socket directly.

```
click → BoardController.sendToServer(MOVE) → GameClient  $\xrightarrow{\text{TCP}}$  GameServer (validate →  
update → check win/draw)  
GameServer  $\xrightarrow{\text{TCP}}$  GameClient → EventBus.post → BoardController.@Subscribe → board  
updates
```

Message protocol.

Dir.	Message	Meaning
C → S	"join"	I want to play.
C → S	MOVE(cell)	Place my symbol on cell (0–8).
C → S	"quit"	Leaving (sent when the window closes).
S → C	WAITING_FOR_OPPONENT	Connected, waiting for a second player.
S → C	GAME_START(symbol,turn)	Game started; you are X/O; is it your turn.
S → C	MOVE(symbol,cell,turn)	A validated move (echoed to both players).
S → C	INVALID_MOVE(text)	Move rejected (not your turn / cell taken).
S → C	GAME_OVER(text)	You win! / You lose! / It's a draw!
S → C	OPPONENT_DISCONNECTED	The other player left.
S → C	GAME_FULL	A game is already in progress.

3. How to run

The two runnable jars are in the repository under `lab7/tictactoe/dist/`. JavaFX 21 needs a Java 17+ runtime. The client jar bundles the JavaFX native libraries for Linux, Windows and macOS, so the same jar runs on all three.

On one computer:

```
# 1) start the server (default port 3000)
java -jar tictactoe-server.jar 3000

# 2) start two clients (two windows)
java -jar tictactoe-client.jar localhost 3000
java -jar tictactoe-client.jar localhost 3000
```

On two different computers:

1. Run the server on one machine and note its IP address (e.g. 192.168.1.20). Make sure both machines are on the same network and the port is open in the firewall.
2. On each computer run a client pointing at the server's IP: `java -jar tictactoe-client.jar 192.168.1.20 3000`

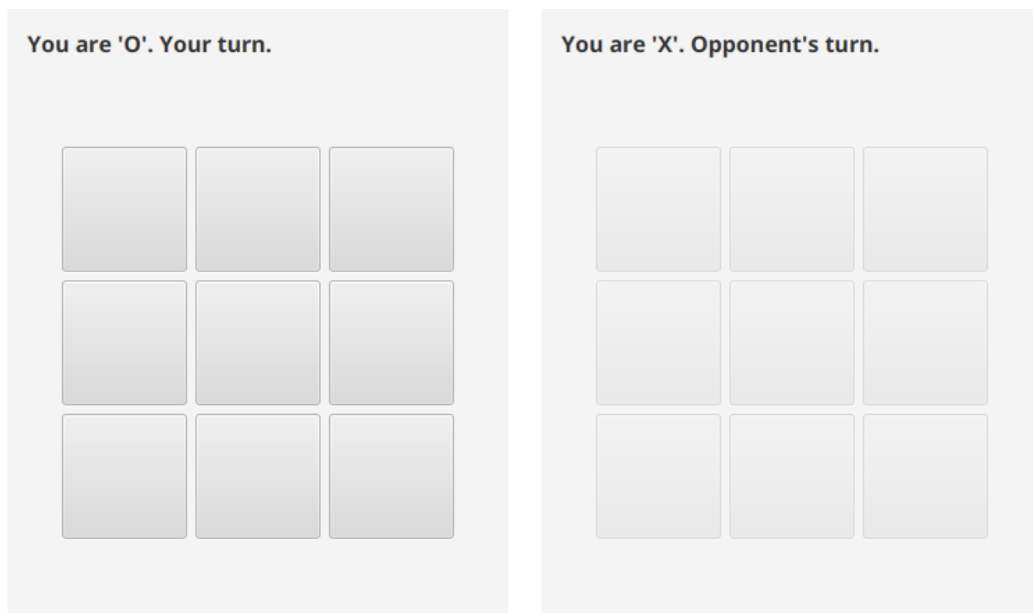
The server listens on port 3000 by default; any port > 1024 can be passed as the first argument to the server and the second argument to each client.

4. Screenshots

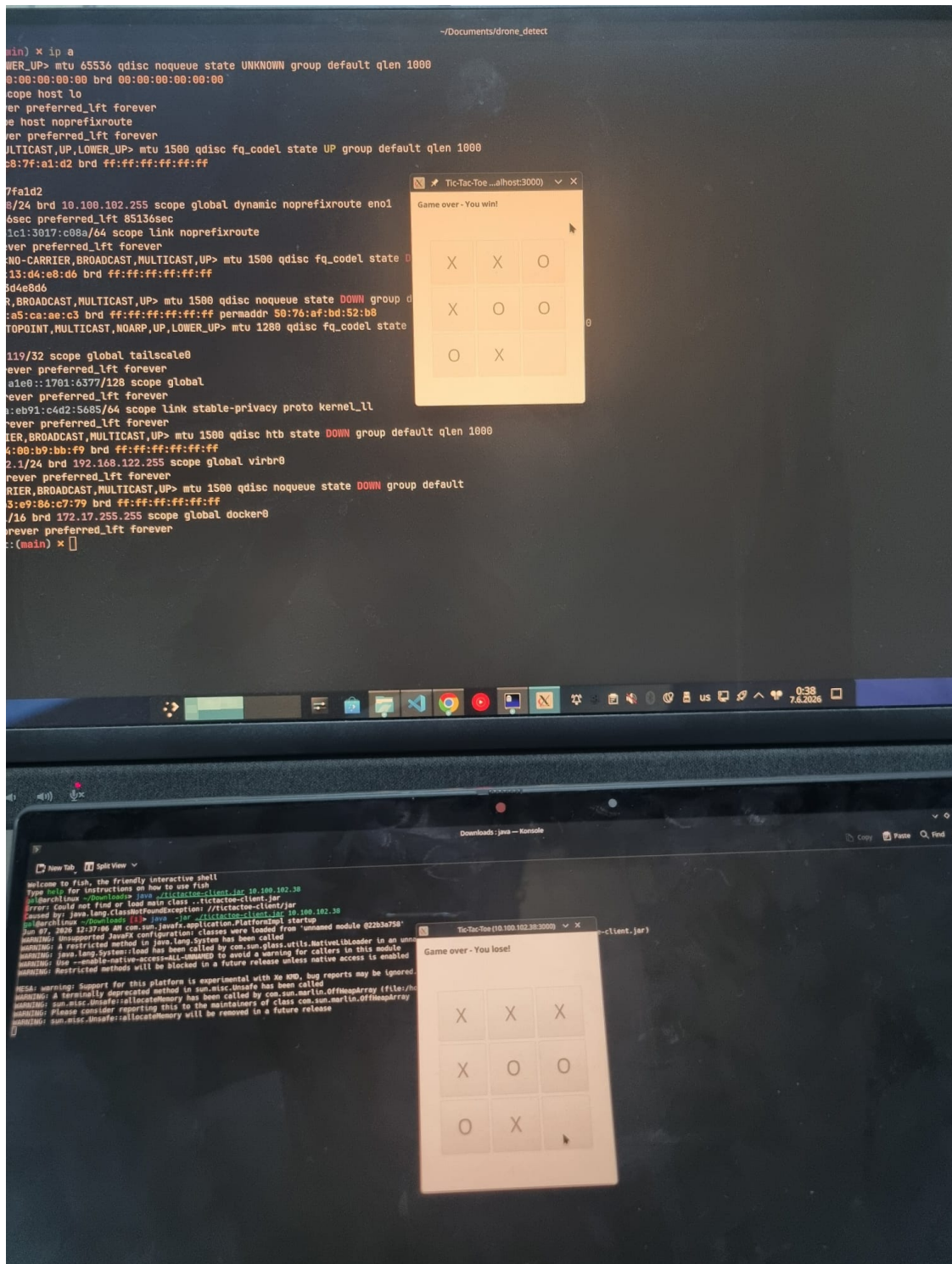
Two clients connected to one server. The server randomly gave one player O and the other X, and told exactly one of them that it is their turn:

Running on two different computers:

The server ran on one machine (IP 10.100.102.38); each computer ran a client connecting to 10.100.102.38:3000 over the network. The top computer won the game (Game over - You win!) while the bottom one lost (Game over - You lose!), showing the same final board on both screens.



Two game clients (left: player O; right: player X).



The game running on two different computers over the network.